Mise en production d'un classifieur d'images satellite

Projet MLOps sur EuroSAT : API à jeton, MLflow, dérive, Terraform et Kubernetes

Adam Beloucif et Emilien Morice

Cours MLOps, Mastère Data Engineering et IA, EFREI Paris, 2026-2027.

Version 1.0.0 du dépôt Adam-Blf/eurosat-mlops, 6 novembre 2026.

Résumé

Nous avons mis en production un classifieur d'images satellite (EuroSAT, 10 classes) pour le cours MLOps. Le modèle, un ResNet18 affiné puis quantifié en int8, est servi par une API FastAPI protégée par jeton, versionné dans MLflow, surveillé par un détecteur de dérive et lancé en conteneurs, sur une VM décrite par Terraform ou sur Kubernetes avec un canary.

Ce rapport suit les niveaux du sujet, donne les mesures obtenues et dit ce qui n'a pas été fait. Aucune VM de production réelle n'a été déployée, le test de charge a tourné contre la stack locale et les manifestes Kubernetes n'ont pas été éprouvés sur un cluster.

97,80 %
accuracy du modèle servi

11,3 Mo
taille du modèle servi

3,34x
plus rapide que le fp32

0
échec sur 7 690 requêtes

Couverture du sujet

Niveau	Exigence	État
0	Stack complète en une commande, en conteneurs	Réalisé, rejoué par la CI
1	Déploiement automatique (commande ou CD)	Écrit et testé, aucune VM réelle
1	Accès refusé sans jeton dans l'en-tête	Réalisé
1	Usage suivi par utilisateur	Réalisé
1	Test de charge	Réalisé, sur la stack locale
2	VM créée par Terraform	Validé, jamais appliqué
2	Interface utilisateur	Réalisé
2	Détection de dérive	Réalisé, démo sur images dégradées
2	MLflow pour versionner et promouvoir	Réalisé
3	Quantization	Réalisé, int8 statique servi
3	Scaling horizontal	Réalisé, mesuré en local
3	Canary sur Kubernetes	Manifestes écrits, non éprouvés

Tableau 1. Niveaux du sujet et état réel.

Sommaire

Contexte et choix du modèle 2
Architecture 3
Niveau 0 : toute la stack en une commande 4
Niveau 1 : jeton, déploiement et test de charge 4
Niveau 2 : Terraform, interface, dérive et MLflow 6
Niveau 3 : quantization, scaling et canary 7
Qualité, observabilité et configuration 9
Choix de sécurité 9
Limites et suites possibles 10
Conclusion 10

Contexte et choix du modèle

Le sujet demande de mettre en production un modèle de machine learning qui ne soit pas trivial, avec quatre niveaux d'exigence, de 0 à 3. Nous avons retenu la classification d'occupation des sols sur le jeu **EuroSAT**. Ce sont des tuiles Sentinel-2 de 64 par 64 pixels, réparties en 10 classes (culture annuelle, culture permanente, pâturage, végétation herbacée, forêt, autoroute, zone industrielle, zone résidentielle, rivière, mer ou lac).

Ce n'est ni MNIST ni CIFAR-10, que le sujet écarte, et le problème n'est pas réglé d'avance. Vu du ciel, une culture annuelle, une culture permanente et un pâturage se ressemblent, et c'est exactement là que le modèle se trompe le plus, comme le montre la matrice de confusion plus bas. Une image satellite se prête aussi à une démonstration de dérive de données, puisqu'un voile de brume ou une surexposition se simule en une ligne.

Le modèle et son entraînement

Le modèle est un **ResNet18** pré-entraîné sur ImageNet, dont la dernière couche est remplacée par une couche à 10 sorties. Les réglages par défaut du script sont AdamW (taux d'apprentissage `1e-3`, décroissance de poids `1e-4`), un ordonnanceur OneCycle, un label smoothing de 0,05 et des augmentations simples, retournements et rotations d'un quart de tour, puisqu'une tuile satellite n'a pas d'orientation privilégiée. Le découpage est stratifié, 70 % pour l'entraînement, 15 % pour la validation et 15 % pour le test, avec une graine fixe (`42`) pour que chaque machine retrouve les mêmes ensembles.

L'historique enregistré pour le modèle livré compte 3 époques, exécutées sur CPU. La précision de validation passe de 93,28 % à 97,33 %. La première époque a duré 1 706 s, les suivantes 377 s et 493 s. La normalisation ImageNet est intégrée au graphe du modèle, qui prend des flottants entre 0 et 1. L'API, la calibration de la quantization et l'entraînement partagent ainsi le même prétraitement, sans le recopier. L'export ONNX s'écarte de PyTorch d'au plus 1.4e-06 sur les logits.

Résultats sur le jeu de test

Sur les 4 050 images de test, le modèle en virgule flottante atteint une accuracy de **97,95** % et un F1 macro de **0,9788**. Les classes les plus faciles sont la forêt et la mer ou le lac. Les plus difficiles sont la culture permanente et la culture annuelle. La confusion la plus fréquente est une culture permanente prise pour une culture annuelle (13 images), suivie de l'inverse (9 images) et de culture annuelle prise pour pâturage (7 images).

Classe	Images de test	F1	Erreurs
Culture annuelle	450	0,957	19
Forêt	450	0,997	1
Végétation herbacée	450	0,981	10
Autoroute	375	0,981	6
Zone industrielle	375	0,988	7
Pâturage	300	0,969	5
Culture permanente	375	0,952	20
Zone résidentielle	450	0,993	3
Rivière	375	0,976	8
Mer ou lac	450	0,994	4

Tableau 2. F1 par classe et nombre d'erreurs du modèle fp32 sur le jeu de test.

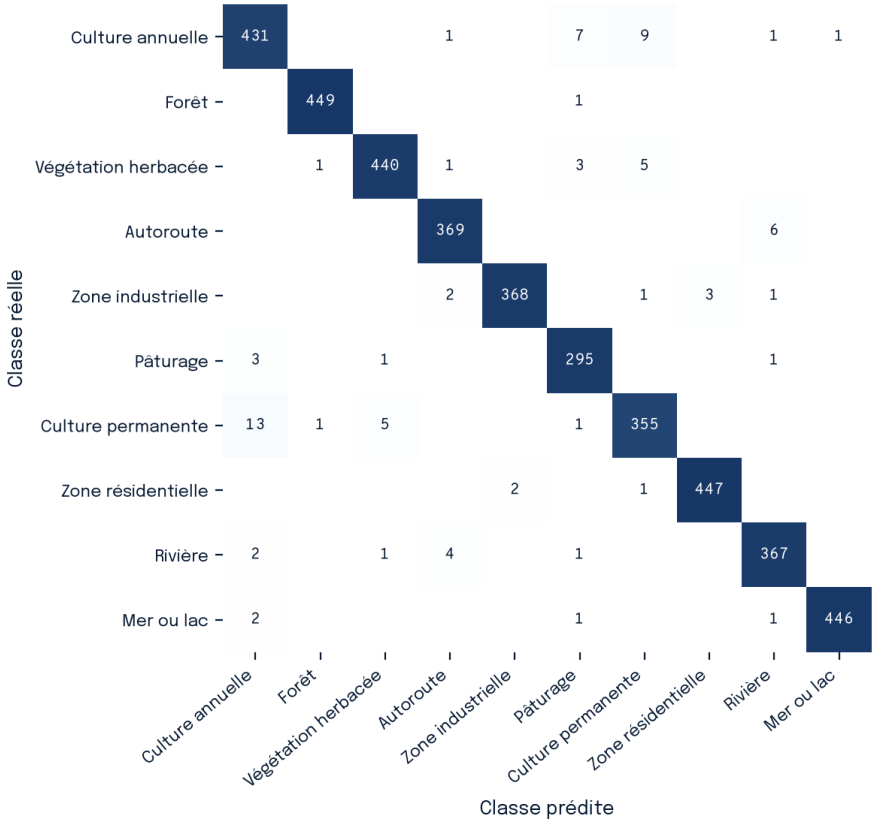


Figure 1. Matrice de confusion du modèle fp32 sur le jeu de test. La teinte indique la part de la classe réelle, le nombre est un effectif d'images.

Architecture

Le navigateur et l'outil de test de charge passent par une **gateway nginx**, qui sert aussi l'interface statique et répartit les requêtes entre les répliques de l'API FastAPI. En production, Caddy se place devant pour le TLS. Les répliques écrivent l'usage dans **Postgres** et exposent leurs métriques à Prometheus. La route de dérive compare le trafic récent aux statistiques d'entraînement.

Le modèle n'est pas copié dans l'image de l'API. Au démarrage, un service **registrar** publie le modèle ONNX quantifié dans **MLflow** et pose l'alias `production` s'il n'existe pas encore. Chaque réplique demande ensuite à MLflow la version désignée par cet alias et la charge à son démarrage. Promouvoir un modèle revient donc à déplacer l'alias, puis à redémarrer les répliques, car aucune ne recharge son modèle à chaud. Hors compose, la variable `MODEL_SOURCE=file` charge directement le

fichier du dépôt.

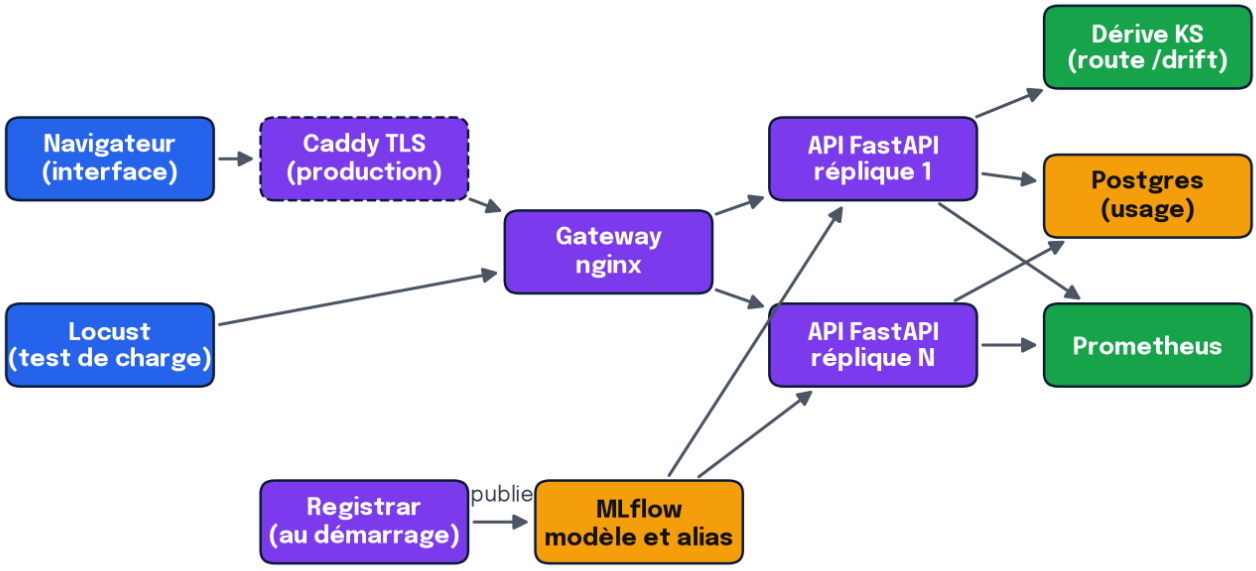


Figure 2. Les services de la stack. Le trait pointillé indique ce qui n'existe qu'en production.

Niveau 0 : toute la stack en une commande

Une seule commande démarre l'ensemble, base de données comprise : `docker compose up --build`. Chaque brique tourne dans son conteneur. Les ports ne sont ouverts que sur `127.0.0.1`, et le fichier `.env` (copié depuis `.env.example`) porte les jetons et le mot de passe de la base.

Service	Rôle	Accès local
postgres	usage par utilisateur, base de MLflow	interne
mlflow	registre de modèles, versions et alias	127.0.0.1:5000
registrar	publie le modèle et pose l'alias production	tâche unique
api	FastAPI, une à plusieurs répliques	interne
gateway	nginx, interface et répartition	127.0.0.1:8080
prometheus	métriques des répliques	127.0.0.1:9090
loadtest	Locust, profil à activer	à la demande

Tableau 3. Services de la stack compose.

L'intégration continue rejoue ce démarrage à chaque pull request. Elle construit toutes les images, lance la stack avec `docker compose up -d --wait`, puis vérifie trois choses au travers de la gateway : une requête sans jeton reçoit un 401, une image de forêt est bien classée `Forest` avec le jeton de démonstration, et la route d'usage connaît l'utilisateur de test. Les identifiants de démonstration sont des valeurs publiques. Le fichier de composition de production refuse de démarrer tant qu'elles n'ont pas été remplacées.

Niveau 1 : jeton, déploiement et test de charge

Accès par jeton et suivi d'usage

Le jeton se place dans l'en-tête `Authorization: Bearer <jeton>` (l'en-tête `X-API-Token` est accepté en repli). Sans jeton, l'API répond 401. Avec un jeton inconnu, elle répond 403. La comparaison se fait en temps constant sur tous les jetons enregistrés, pour qu'un temps de réponse ne révèle rien. Les jetons viennent de la variable `API_TOKENS` sous la forme `utilisateur:jeton`, et une liste vide refuse

tout le monde. Un jeton administrateur distinct ouvre `/drift` et la vue complète de `/usage`.

Chaque appel est écrit en base (date, utilisateur, route, code de réponse, classe, confiance, latence et cinq statistiques de l'image), ce qui sert à la fois au suivi d'usage et à la détection de dérive. Une limite de débit par jeton, 1 200 requêtes par minute par défaut, protège l'API. Elle est tenue en mémoire par réplique, si bien que la limite réelle monte avec leur nombre. Les envois sont bornés à 2 Mo et aux formats JPEG, PNG, WEBP et TIFF.

Route	Accès	Rôle
GET /health, /ready	libre	sondes de vie et de disponibilité
POST /predict	jeton	classe, probabilités, latence
GET /model	jeton	version et alias du modèle chargé
GET /usage	jeton	appels par utilisateur, tous pour l'admin
GET /drift	jeton admin	verdict KS sur la fenêtre récente
GET /metrics	réseau interne	métriques Prometheus

Tableau 4. Routes de l'API et niveau d'accès.

Déploiement automatique

Le déploiement tient dans une commande, `python scripts/deploy.py --version 1.0.0`, pilotée par `DEPLOY_HOST` et `DEPLOY_ENV_FILE`. Le script refuse un fichier d'environnement incomplet ou qui contient encore des valeurs de démonstration. Il écrit le `.env` distant en mode 600 sans jamais l'afficher, lance la stack de production, puis attend la sonde de santé. L'option `--dry-run` affiche le plan sans contacter personne.

Côté pipeline, le workflow `release.yml` se déclenche sur un tag `vX.Y.Z`. Il vérifie que le tag correspond à `pyproject.toml`, publie les trois images sur GHCR avec le numéro de version et le commit (jamais `latest`), puis crée la release avec les notes tirées du CHANGELOG. Le workflow `deploy.yml` prend le relais à la publication de la release, derrière un environnement `production` où l'on peut exiger une validation humaine.

Ce que nous n'avons pas fait. Aucune VM de production réelle n'a été déployée. Le job de `deploy.yml` est ignoré tant que la variable `DEPLOY_ENABLED` n'est pas à `true`, ce qui garde un dépôt sans VM au vert. Le script est couvert par des tests unitaires et par son mode `--dry-run`, mais la séquence complète sur une machine distante n'a pas été jouée.

Test de charge

Le scénario Locust simule un analyste qui envoie l'une des dix tuiles d'exemple (une par classe) à `/predict`, en consultant de temps en temps sa propre consommation sur `/usage`, avec une pause de 0,05 à 0,3 s entre deux actions. Les jetons viennent de l'environnement et ne sont jamais affichés. Un garde-fou refuse toute cible qui n'est pas locale, afin de ne jamais tirer sur une production par erreur.

Le test a tourné contre la stack compose locale, pas contre une production. Nous avons joué le même scénario avec 24 utilisateurs simulés et 1, 2 puis 4 répliques de l'API, limiteur de débit désactivé pour la durée de la mesure. Sur les 7 690 requêtes des trois paliers, aucune n'a échoué.

Répliques	Requêtes	Échecs	req/s	Médiane (ms)	p95 (ms)	p99 (ms)
1	2 288	0	53,0	210	380	550
2	2 516	0	59,2	140	370	500
4	2 886	0	67,7	95	320	490

Tableau 5. Test de charge sur la route /predict selon le nombre de répliques de l'API.

Passer de 1 à 4 répliques fait monter le débit de 53,0 à 67,7 requêtes par seconde (+28 %) et baisse la latence médiane de 210 à 95 ms (-55 %). Le gain est modeste. Le générateur de charge, la gateway, la

base et les répliques partagent les mêmes coeurs de la même machine. Nous n'avons pas mesuré la répartition du CPU, c'est l'explication que nous retenons. Un test sur une VM dédiée, avec le générateur sur une autre machine, dirait davantage.

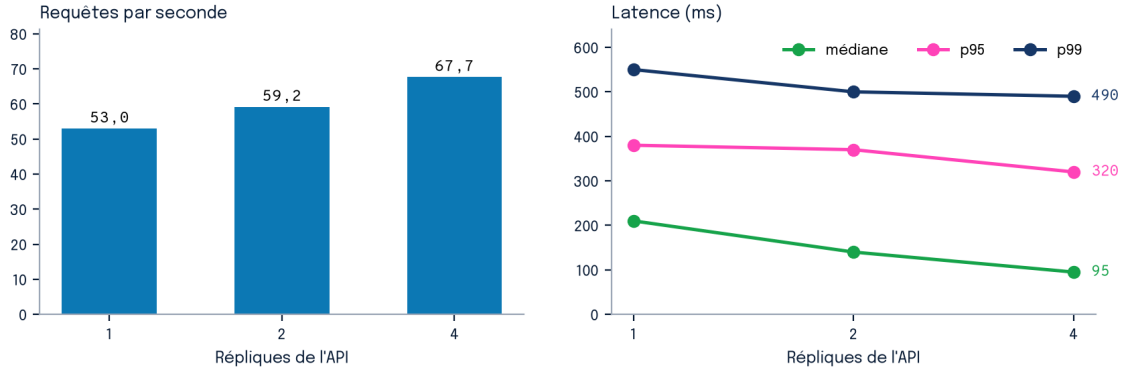


Figure 3. Débit et latences selon le nombre de répliques.

Niveau 2 : Terraform, interface, dérive et MLflow

Terraform

Le dossier `infra/terraform` décrit une VM Hetzner Cloud sous Ubuntu 24.04 où cloud-init installe Docker et le plugin compose, un pare-feu côté fournisseur et la clé SSH de déploiement. Le pare-feu n'ouvre le SSH qu'aux adresses de `admin_cidrs`, et une validation refuse `0.0.0.0/0`. L'utilisateur `deploy` n'est pas root et n'accepte que la clé, comme le suppose `scripts/deploy.py`. `ignore_changes` sur `user_data` évite qu'une retouche de cloud-init détruise une VM en marche.

La configuration passe terraform fmt -check et terraform validate, mais elle n'a jamais été appliquée, parce qu'un `apply` crée une ressource payante (environ 4 euros par mois pour un cx22). L'état est local et ignoré par git. Au-delà d'une démonstration d'école, il faudrait un backend distant.

Interface web

L'interface est une page statique servie par la gateway. On y glisse une tuile, on colle son jeton, et elle affiche la classe en français avec la confiance. Les erreurs 401, 403, 413, 415 et 429 ont chacune un message lisible. Rien ne vient d'un CDN : les icônes et les polices (Epilogue et Overpass Mono, les mêmes que ce rapport) sont embarquées, et la politique de sécurité de contenu n'autorise que l'origine de la page.

Détection de dérive

La référence est un sous-échantillon de 2 000 images d'entraînement, résumées par cinq statistiques : luminosité, moyenne de chaque canal rouge, vert et bleu, et contraste. La route `/drift` compare la fenêtre des images récentes (200 par défaut) à cette référence avec un test de Kolmogorov-Smirnov à deux échantillons par statistique. Le seuil de 0,01 est divisé par le nombre de statistiques, 5, d'après la correction de Bonferroni, soit 0,002. Sous 30 images, le détecteur ne rend aucun verdict. Le résultat alimente aussi des jauges Prometheus.

La démonstration tire 300 images du jeu de test, une première fois intactes, une seconde fois dégradées par un voile de brume et une surexposition (chaque pixel est multiplié par 0,6 puis augmenté de 95). Sur le trafic intact, aucune statistique n'est signalée, la plus basse p-value étant 0,15. Sur le trafic dégradé, toutes les statistiques (5 sur 5) sont signalées, avec des p-values d'au plus 3,4e-76.

Statistique	Réf.	Intact	KS	p-value	Dégradé	KS	p-value
luminosité	0,378	0,374	0,043	0,704	0,596	0,806	4,8e-177
moyenne rouge	0,345	0,342	0,040	0,774	0,576	0,717	6,1e-133
moyenne verte	0,381	0,378	0,033	0,927	0,598	0,830	7,7e-191
moyenne bleue	0,408	0,403	0,050	0,516	0,613	0,888	1,2e-230
contraste	0,095	0,093	0,070	0,152	0,056	0,558	3,4e-76

Tableau 6. Test KS sur 300 images, trafic intact puis dégradé. Réf. désigne la moyenne de la référence d'entraînement, toutes les moyennes sont comprises entre 0 et 1.

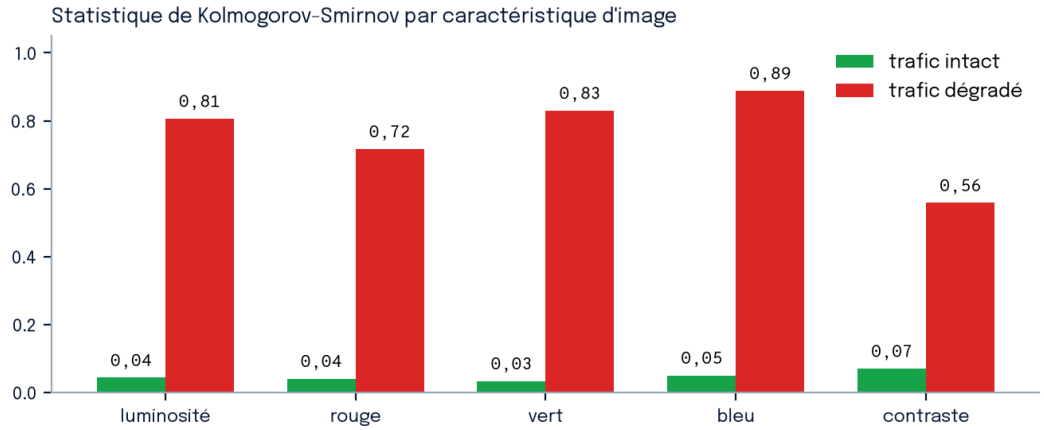


Figure 4. Statistique KS par caractéristique, trafic intact et dégradé.

Cette démonstration est volontairement grossière. Elle montre que le détecteur reste silencieux sur un trafic normal et réagit à un décalage massif, pas qu'il repère des dérives fines. Il ne regarde que des statistiques de couleur. Un changement de saison ou de région qui garderait la même luminosité moyenne passerait inaperçu.

MLflow : versions et alias

Le module `registry.py` s'appuie sur les alias de MLflow. La commande `publish` journalise le modèle ONNX, les statistiques de référence et les métriques dans un run, enregistre une nouvelle version de `eurosat-classifier` et lui pose l'alias `candidate`. La commande `promote` déplace l'alias `production` vers cette version, à condition que son accuracy de test ne soit pas inférieure de plus de 0,005 à celle de la version en place, sauf avec `--force`. Le service `registrar` de la stack appelle `bootstrap`, qui publie puis promeut seulement si aucun alias `production` n'existe, ce qui rend le démarrage répétable sans empiler de versions.

Niveau 3 : quantization, scaling et canary

Quantization

Nous avons comparé trois variantes ONNX du même modèle. La quantization **statique** (format QDQ, poids int8 par canal, activations uint8) est calibrée sur 256 images d'entraînement. La quantization **dynamique** ne convertit que les poids en int8 et calcule les échelles des activations à chaque appel. Les trois sont évaluées sur les 4 050 images de test. La latence est mesurée en batch 1 sur un seul fil, médiane et p95 sur 200 appels après 20 appels d'échauffement.

Modèle	Taille (Mo)	Accuracy	F1 macro	Médiane (ms)	p95 (ms)	Accél.
fp32	44,72	0,9795	0,9788	8,12	40,32	1,00x
int8 dynamique	11,24	0,9798	0,9791	181,19	231,38	0,04x
int8 statique	**11,30**	**0,9780**	**0,9773**	**2,43**	**5,53**	**3,34x**

Tableau 7. Les trois modèles sur le jeu de test. Le modèle servi est l'int8 statique (en gras).

Les deux variantes divisent la taille du fichier par 4,0. Côté qualité, la version statique classe correctement 6 images de moins que le fp32 sur 4 050, la dynamique en classe 1 de plus. Ces écarts sont de l'ordre de quelques images et nous ne les lisons ni comme une perte ni comme un gain réels.

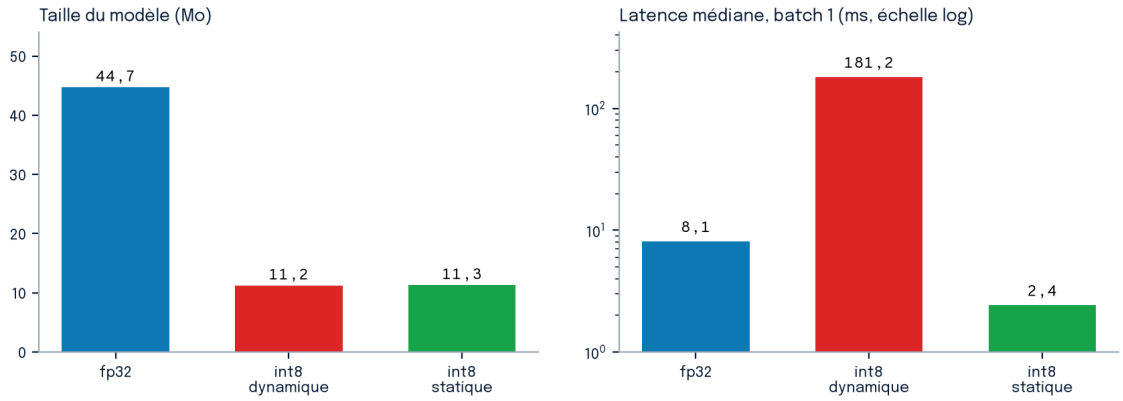


Figure 5. Taille et latence médiane des trois modèles.

Côté vitesse, les deux variantes ne se comportent pas du tout pareil. La version statique est 3,34 fois plus rapide que le fp32, et réduit aussi l'écart entre la médiane et le p95. La version dynamique est 22 fois **plus lente** que le fp32 (médiane de 181 ms). Nous n'avons pas profilé son graphe. L'explication la plus plausible est que, sur CPU, les convolutions d'un ResNet ne trouvent pas de noyau int8 efficace en quantization dynamique, alors qu'il faut en plus calculer les échelles des activations à chaque appel. En statique, les échelles sont figées à la calibration et le runtime enchaîne des opérations int8. On retient qu'une taille réduite ne garantit pas un modèle plus rapide, et que cela se mesure sur la cible. C'est le modèle statique qui est servi.

Scaling horizontal

Le nombre de répliques de l'API se règle avec `API_REPLICAS` (2 par défaut). La gateway nginx s'appuie sur le DNS de Docker, qui renvoie toutes les répliques, avec un délai de résolution de 5 secondes pour suivre celles qui apparaissent ou disparaissent. Prometheus interroge chaque réplique directement, et l'usage va dans une base Postgres commune, de sorte que les décomptes par utilisateur restent cohérents d'une réplique à l'autre. Les mesures du niveau 1 (1, 2 et 4 répliques) sont celles de ce scaling, avec la réserve déjà dite sur la machine partagée.

Canary sur Kubernetes

Le dossier `k8s` contient une base kustomize (namespace, Postgres, MLflow, job du registre, API, gateway, ingress) et deux overlays. L'overlay `local` déploie la stack complète. L'overlay `canary` ajoute un déploiement `api-canary` d'un seul pod qui charge l'alias `candidate` au lieu de `production`, avec son propre service et un second ingress.

Le partage du trafic repose sur les annotations de canary d'ingress-nginx. Un poids de 10 envoie environ 10 % des requêtes vers le candidat, et tout appel portant l'en-tête `X-Canary: always` le joint à coup sûr, ce qui permet de tester un candidat à la main. Quand il est validé, on déplace l'alias `production` dans MLflow, on redémarre les répliques et on retire l'overlay. Les pods tournent en non-root, système de fichiers en lecture seule, toutes les capacités retirées, avec des sondes `/ready` et `/health`.

Ces manifestes n'ont pas été éprouvés sur un cluster. Aucun résultat du dépôt ne documente une exécution sous Kubernetes, et la bascule se fait à la main, sans analyse automatique des métriques du candidat.

Qualité, observabilité et configuration

L'intégration continue tourne à chaque pull request et sur `main`. Un premier job exécute `ruff check` et `ruff format --check`. Un deuxième lance `pytest`, dont les tests d'authentification, d'usage, de dérive, de déploiement et du garde-fou de test de charge. Un troisième construit toutes les images et rejoue le démarrage et le test de fumée décrits au niveau 0. Le script `sync_version.py` vérifie que le badge du README, le CHANGELOG et `pyproject.toml` annoncent la même version.

Chaque réplique expose cinq séries Prometheus : les requêtes par utilisateur, route et code, les prédictions par classe, un histogramme du temps d'inférence (de 2 ms à 1 s), la p-value de dérive par statistique et un indicateur de dérive détectée. Prometheus découvre les répliques par le DNS de Docker. La route `/metrics` n'est pas joignable par la gateway publique.

Variable	Rôle	Défaut
API_TOKENS	jetons clients, utilisateur:jeton séparés par des virgules	vide, tout refusé
ADMIN_TOKEN	jeton admin pour /drift et /usage global	vide
API_REPLICAS	nombre de répliques de l'API	2
MODEL_SOURCE	file ou mlflow	file (mlflow en compose)
MODEL_ALIAS	alias MLflow chargé au démarrage	production
RATE_LIMIT_PER_MINUTE	limite par jeton et par réplique	1200
MAX_UPLOAD_BYTES	taille maximale d'une image	2 000 000
DRIFT_WINDOW	images récentes comparées à la référence	200

Tableau 8. Principales variables d'environnement.

Choix de sécurité

Nous avons traité la sécurité à chaque couche plutôt qu'en fin de projet. Le tableau résume les choix et l'endroit où ils vivent.

Sujet	Choix	Où
Accès	jeton obligatoire, 401 sans jeton, 403 si inconnu, comparaison en temps constant	api/auth.py
Fichiers envoyés	formats en liste blanche (SVG exclu), 2 Mo, 25 millions de pixels au plus	preprocess.py, nginx
Secrets	dans des fichiers ignorés par git et dans les secrets GitHub, jamais dans le dépôt. Le déploiement refuse les valeurs de démonstration	scripts/deploy.py
Conteneurs	système de fichiers en lecture seule, capacités retirées, non-root, ports sur 127.0.0.1	docker-compose.yml
Entrée web	en-têtes nosniff, DENY, CSP limitée à l'origine, limite de débit, route de métriques masquée, CORS restreint, TLS par Caddy en production	gateway, Caddyfile
Test de charge	refus de toute cible non locale et du mode interface web	loadtest/guard.py
Chaîne de livraison	actions GitHub épinglées par empreinte, images versionnées, jamais latest	workflows
Réseau cloud	pare-feu fournisseur, SSH limité à admin_cidrs, connexion par clé	infra/terraform

Tableau 9. Mesures de sécurité du projet.

Limites et suites possibles

Ce que le projet ne prouve pas, et ce que nous ferions ensuite.

- **Pas de production réelle.** Terraform est validé mais jamais appliqué, et le workflow de déploiement est resté inactif. La chaîne VM, déploiement, test n'a pas été exécutée de bout en bout. La suite logique est d'appliquer Terraform pour une séance, de déployer, puis de détruire.
- **Charge mesurée en local.** Les chiffres du scaling viennent d'une seule machine partagée avec le générateur de charge. Ils comparent des configurations entre elles mais ne disent rien d'une production. Nous les refairions sur une VM dédiée.
- **Entraînement court.** 3 époques sur CPU, un seul découpage et une seule graine, sans intervalle de confiance. Nous n'avons pas comparé d'autres architectures. Un entraînement sur GPU avec plusieurs graines serait plus solide (le notebook Colab permet de s'entraîner sur GPU).
- **Dérive limitée aux couleurs.** Le détecteur ne regarde ni les prédictions ni les étiquettes. En vrai, il faudrait suivre la distribution des classes prédites et relier la dérive à un réentraînement.
- **Limiteur de débit par réplique.** Il vit en mémoire, il faudrait le sortir vers un stockage partagé. De même, stocker des empreintes de jetons en base serait mieux que la variable d'environnement.
- **Canary à la main.** Les manifestes ne sont pas testés sur un cluster et rien ne bascule ni ne revient en arrière automatiquement. On pourrait brancher la décision sur les métriques Prometheus.
- **Quantization dynamique.** Notre explication de sa lenteur reste une hypothèse. Un profilage avec ONNX Runtime la confirmerait ou la corrigerait.

Conclusion

Nous avons mis en production un classifieur d'images satellite qui tient dans une seule commande, derrière une API à jeton avec suivi d'usage, un registre de modèles, un détecteur de dérive et une interface. Le modèle servi pèse 11,3 Mo pour une accuracy de 97,80 % sur le jeu de test.

Le plus instructif a été de mesurer plutôt que de supposer. La quantization dynamique, qui réduit la taille aussi bien que la statique, ralentit le modèle d'un facteur 22, et passer à quatre répliques ne gagne qu'une partie de ce que l'on attendait. Les deux résultats nous ont appris à garder une mesure derrière chaque choix, et à écrire clairement ce qui n'a pas été fait.